TurboHap: accelerating FindHap haplotype phasing and genotype imputation while preserving output compatibility

Adrian R. Carr[1]

[1]Green Neuron AI, Edinburgh, UK

Abstract

Genotype imputation and haplotype phasing are routine steps in livestock genomic evaluation, but processing large populations can be costly. TurboHap reimplements FindHap v3 in C++17, retaining its input formats, options and inference procedure while reducing repeated work through compact storage, early rejection of incompatible haplotypes and parallel processing. A low-memory mode, TurboHap LM, uses temporary files to reduce resident memory. Both modes were compared with the original Fortran baseline on seven simulated datasets containing 10,000–1,000,000 genotyped animals and 1,000–100,000 loci, with two runs per program using ten threads on one ARM64 system. Median speedups were 10.0–93.4× for standard TurboHap and 7.1–56.2× for LM. On the million-animal dataset, standard mode reduced runtime from 6 h 53 min 40 s to 4 min 26 s with 2.2% more peak resident memory, whereas LM took 7 min 22 s and used 25.1% less. Across all seven workloads, result files matched Fortran byte for byte except for permitted crossover-record reordering, and the two C++ modes matched exactly. These results show that substantial implementation-level acceleration is possible without changing FindHap's outputs on the tested inputs.

Keywords: genotype imputation; haplotype phasing; livestock genomics; FindHap; C++; parallel computing; memory efficiency

Correspondence: adrian.carr@greenneuron.ai

Introduction

Livestock genomic evaluations combine animals genotyped on marker panels of different densities. Imputation fills the gaps between these panels, using shared haplotypes and pedigree relationships. FindHap combines these two sources of information and has been evaluated for genomic prediction and high-density genotypes [1,2]. As databases grow, the time and memory needed for imputation become increasingly important.

For established FindHap workflows, acceleration is most useful when existing parameter files, parsers and downstream analyses continue to work. Comparable average imputation accuracy is insufficient if individual calls change. FindHap's iterative matching makes this a demanding implementation problem: the first acceptable haplotype, frequency ties and library updates can influence subsequent assignments, so changing candidate order can alter results beyond the current animal.

TurboHap addresses this problem by reimplementing FindHap v3 [3] while retaining its inference procedure and output conventions. The earlier FindHap papers describe the method; the 2013 study evaluated version 2, whereas v3-specific behaviour follows the software documentation and reference code [1–3]. Other livestock imputation methods also exploit family relationships [4]. The contribution here is faster execution of an established method, with compatibility assessed through its result files.

This study describes the implementation and compares its output, runtime and peak resident memory with the original Fortran baseline across seven simulated workloads. A low-memory mode provides an alternative when RAM is limited.

Materials and methods

FindHap and output compatibility

FindHap divides chromosomes into segments and builds a library of haplotypes for each segment. It searches for a compatible pair for each animal: the first match constrains the complementary haplotype (Figure 1A). Repeated passes refine the library and the assignments. Pedigree information helps resolve inheritance, phase orientation and crossovers, and can reconstruct haplotypes of ungenotyped ancestors [1,3]. TurboHap retains the configured iterations, segment-length steps and option-dependent behaviour of v3 (Figure 1B).

The reference is the original FindHap v3 Fortran source adapted to compile with gfortran on ARM64. Comparisons use the same inputs, options, explicit thread counts and C locale, with a fresh directory for each run. All enabled result files are compared, including their formatting; console progress and timing messages are excluded.

All files are compared byte for byte except `cross.overs`: multithreaded Fortran can write the same crossover records in a different order. For this file, we preserve the header and sort complete records before comparison, retaining every field and duplicate. The two TurboHap modes must match exactly, including crossover order.

Compact genotype storage

Both modes parse input records once and cache each at its declared chip length. Ordinary genotype states require two bits

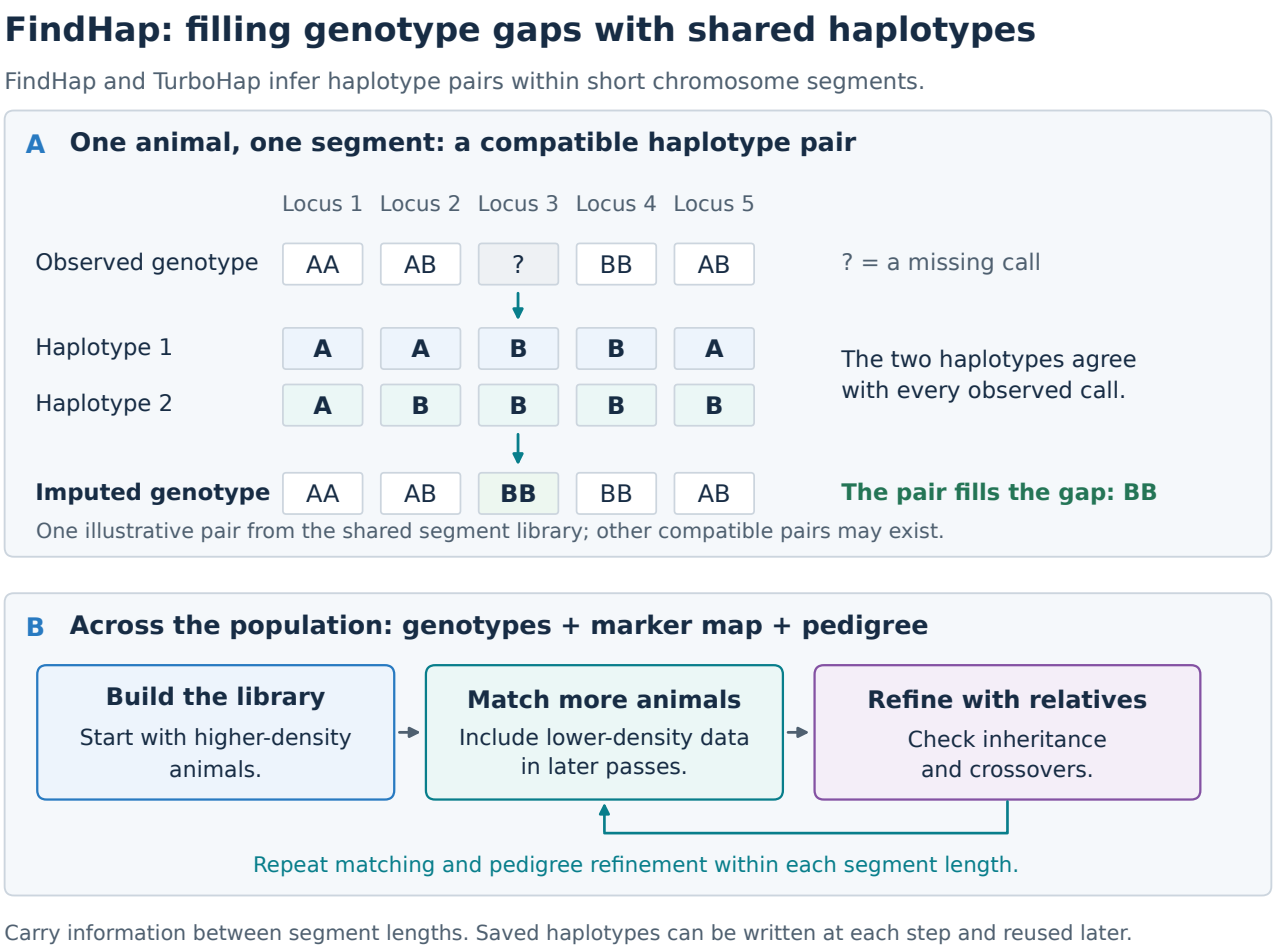


Figure 1. Reconstructing missing genotypes from shared haplotypes. (A) A compatible haplotype pair explains an animal's observed genotypes within a chromosome segment. In this example, both selected haplotypes carry B at locus 3, filling the missing genotype as BB. The illustrated pair is not necessarily the only compatible pair. A and B denote alleles; AB is heterozygous and ? is missing. (B) Higher-density animals establish the segment library before lower-density animals enter later passes. Matching and pedigree refinement repeat, with information carried between segment lengths. Saved haplotypes can be written at each step and reused in subsequent runs.

per locus; records containing other values retain byte storage (Figure 3A). This saves space without changing the decoded values or the interpretation of blank fields.

Packing mutable genotypes also requires care at segment boundaries: workers updating adjacent segments must not modify different bits of the same byte. TurboHap therefore packs working genotypes separately by segment, aligning each start to a byte boundary. Workers decode the segment they need and save edits for later passes; layouts with overlapping prior segments instead use dense storage to preserve shared-locus updates.

Haplotype identifiers use 16- or 32-bit storage according to the configured capacity. When segment lengths change, only the required part of the previous library is retained, with identifiers preserved. Supplementary File 1 gives the storage rules and thresholds.

Faster haplotype search

FindHap scans an ordered haplotype library for the first acceptable candidate. Accelerating this search requires preserving both that choice and the frequency information gathered while rejecting earlier candidates, because this information affects later ordering.

For each candidate, two bit masks summarise known alleles at up to 64 positions across the segment (Figure 2A). Conflicts at these sampled positions give a lower bound on the full conflict count. If that bound exceeds the allowed tolerance, the candidate cannot match and can be skipped. Every survivor undergoes the complete acceptance check, and the summaries are updated when haplotypes change. The example in Figure 2B,C shows how filtering reduces the number of detailed checks while preserving the selected match.

For eligible searches allowing no conflicts, standard TurboHap uses an ordered bitmap index to find surviving candi-

dates without visiting each rejected entry. A range-minimum tree retrieves the lowest frequency and earliest tied position in the skipped prefix, preserving the information that Fortran obtains during its scan. Small libraries and unsupported search conditions use the compact scan; the activation rules are given in Supplementary File 1.

Complete conflict checks compare several loci at once using ARM NEON, or a portable SIMD-capable loop on other systems. Checks stop once the accumulated count exceeds the tolerance. A zero-conflict result can also be reused while collecting missing loci; candidates with conflicts retain the original position-dependent checks.

Parallel processing without changing results

OpenMP distributes independent population segments among workers while preserving animal order within each segment. Pedigree processing also retains animal order because one animal's updates can affect the next. A persistent team processes segment work for successive animals, synchronising before the updates that depend on it. Keeping this team active avoids repeated parallel-region setup (Figure 3C).

Workers also reconstruct and format output in bounded batches. A single writer emits the completed records in animal order. The benchmarks include all output writing, so the speedups do not depend on suppressing result files.

Low-memory execution

TurboHap LM uses the same inference code with temporary files and bounded genotype buffers (Figure 3B). A worker processes consecutive animal blocks within its segment, saving genotype edits before replacing a block. Haplotype libraries and assignments remain in RAM. LM uses the compact candidate scan rather than the additional ordered bitmap index.

Both modes are included in one executable; `--low-memory` selects LM before data allocation. Buffer sizes, dense fallbacks and temporary-file handling are described in Supplementary File 1.

Synthetic datasets

Seven datasets were generated with USDA genosimV4 [5]. Six combine 10,000 or 100,000 genotyped animals with 1,000, 10,000 or 100,000 markers; the seventh has 1,000,000 animals and 10,000 markers. Each pedigree includes four additional ungenotyped founder females. Reported marker counts exclude 100 simulated quantitative trait loci.

Datasets were simulated separately, using eight initial ancestors and 30 chromosomes, with chromosome 30 treated as X. Approximately 83.33% of animals have the full panel and 16.67% have every third marker. The pedigrees contain unusually large families, including a parent with 499,998 progeny in the largest dataset. These workloads test different input sizes under a simple simulation regime; they do not represent the range of livestock pedigrees. The generation recipe and settings are given in Supplementary File 1.

All programs used four iterations, three segment-length steps, maximum and minimum lengths of 100 and 50, all four output selectors set to 1, and an error rate of 0.004. No prior haplotypes were supplied. Haplotype capacity (`maxhap`) was 50,000 for the six smaller datasets and 500,000 for the million-animal dataset. The largest case therefore changes capacity and identifier width as well as animal count.

Preparation assigned parental sex consistently with sire and dam roles before genotype simulation, and aligned the marker-location arithmetic of the two simulators. Before timing, validation of the retained truth checked marker coverage, phase–genotype consistency and male X-chromosome haploidy. Input and truth checksums are supplied in Supplementary File 2.

Measurement and validation

Measurements used ten OpenMP threads on an NVIDIA DGX Spark with 20 logical CPU cores, Ubuntu 24.04 and GCC/gfortran 13.3.0 on ARM64. The GPU was unused. Fortran used `-O3 -fopenmp -fno-underscoring`, whereas the C++ build added native CPU tuning, loop unrolling and link-time optimisation to `-O3` and OpenMP. This compares the specified builds, rather than isolating implementation changes from compiler tuning. Exact commands and the Fortran reference sources are provided in Supplementary File 2.

Elapsed time covers the complete executable: input, inference, output and LM temporary-file I/O. Output comparison follows timing. Memory is peak process resident set size (RSS), recorded by GNU `time` and reported in GiB. RSS excludes temporary-disk capacity and does not capture all filesystem-cache memory.

Each program was run twice per dataset on 21–22 October 2026, in the order Fortran, TurboHap, LM, LM, TurboHap, Fortran. There were no warm-up runs or forced cache eviction. The server also ran other work, whose load was not systematically recorded. We report median runtime and median RSS separately; with two observations, each is the midpoint. Speedup is the ratio of runtime medians, and memory change is $100 \times$ (C++ median RSS / Fortran median RSS $- 1$).

Both C++ modes used the same executable, built from source files identical to the v1.0.0 release. Supplementary File 2 records source and executable hashes. The bundled installation check runs the original FindHap example in both modes at one and four threads, comparing outputs with saved Fortran hashes.

AI assistance

Adrian R. Carr conceived and led the project, directing its methods, validation and interpretation. Under his direction, OpenAI ChatGPT and Codex assisted with software development, testing, analysis, figures and manuscript preparation. The author retains responsibility for the accuracy, integrity and originality of the work.

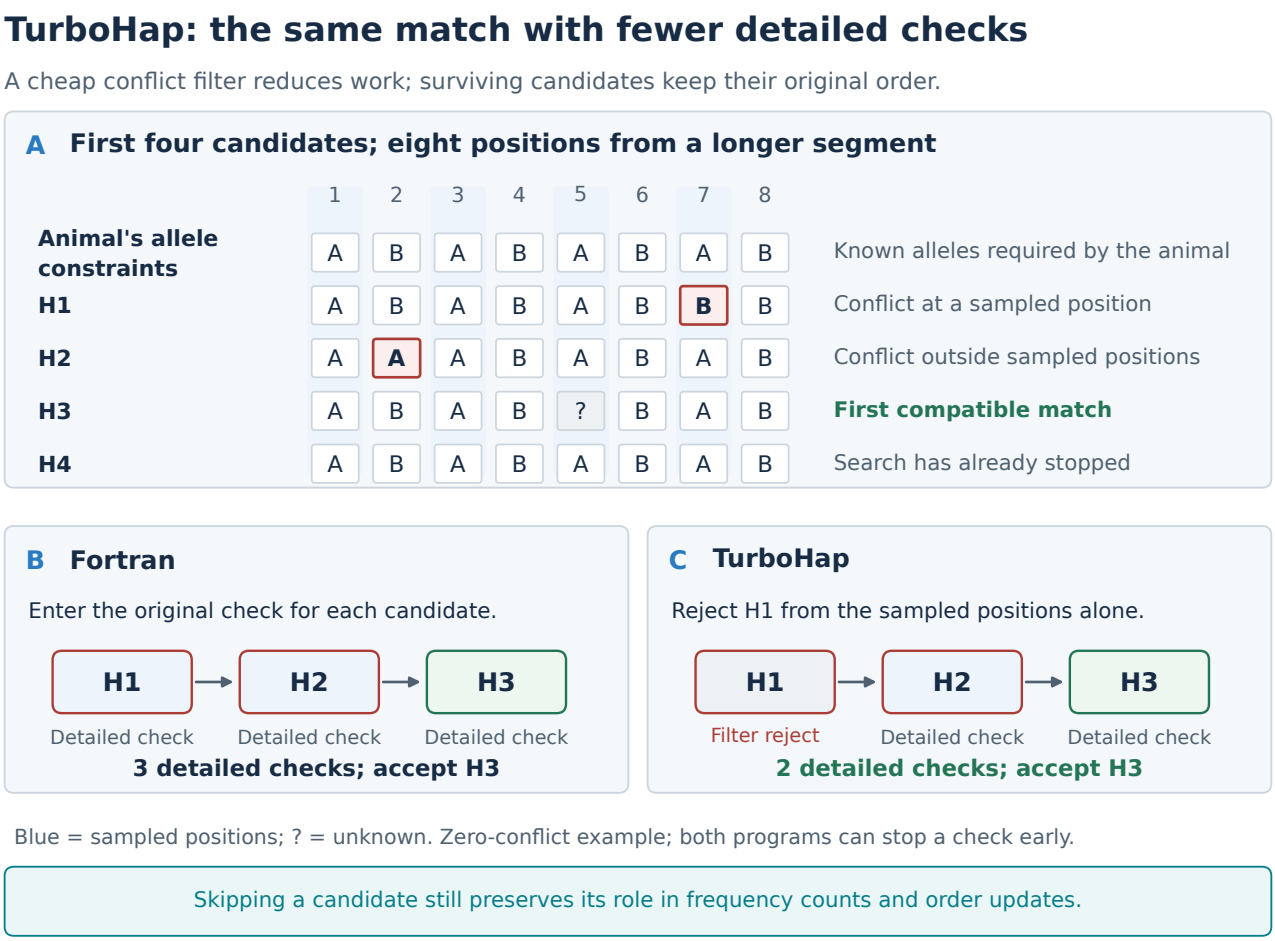


Figure 2. Rejecting incompatible candidates before detailed checking. (A) Eight positions illustrate constraints on one haplotype and four candidates in their current search order. Blue columns are sampled by the filter. H1 conflicts at a sampled position; H2 conflicts outside those positions; H3 is compatible. Unknown alleles (?) create no conflict. (B) Fortran enters its detailed acceptance check for H1, H2 and H3. (C) TurboHap rejects H1 using the sampled positions and enters detailed checks for H2 and H3. Both select H3 and preserve the required frequency and ordering updates. This zero-conflict example assumes compatible unshown positions. Both programs can stop detailed checks early; the illustrated counts are not runtime measurements.

Results

End-to-end runtime

Across all seven datasets, standard TurboHap was the fastest program, achieving median speedups of 10.0–93.4× over Fortran; LM also reduced runtime substantially, with speedups of 7.1–56.2× (Table 1).

For 1,000,000 animals and 10,000 loci, median runtime fell from 6 h 53 min 40 s with Fortran to 4 min 26 s with TurboHap and 7 min 22 s with LM. At 100,000 animals and 100,000 loci, the corresponding runtimes were 1 h 17 min 0 s, 2 min 34 s and 3 min 57 s, giving speedups of 30.0× and 19.5×.

Peak resident memory and compatibility

Standard TurboHap reduced median peak RSS by 7.2–40.7% on the six smaller datasets, but used 16.472 GiB versus 16.117 GiB for Fortran on the million-animal dataset, an increase of 2.2% (Table 2). This largest case also raises configured capacity above the 65,535 threshold, so assignment and progeny haplotype identifiers require 32 bits rather than 16. Those arrays therefore lose their storage advantage over Fortran, while standard mode retains its additional search indexes. The measurements do not isolate the contribution of each allocation. LM had the lowest peak RSS throughout, using 16.3–44.5% less than Fortran; its million-animal peak was 12.077 GiB, a reduction of 25.1%.

All 28 C++ runs passed comparison with the Fortran reference, allowing only the crossover ordering difference described above. All four C++ runs per dataset matched one another exactly. All 42 timing records reported zero swaps.

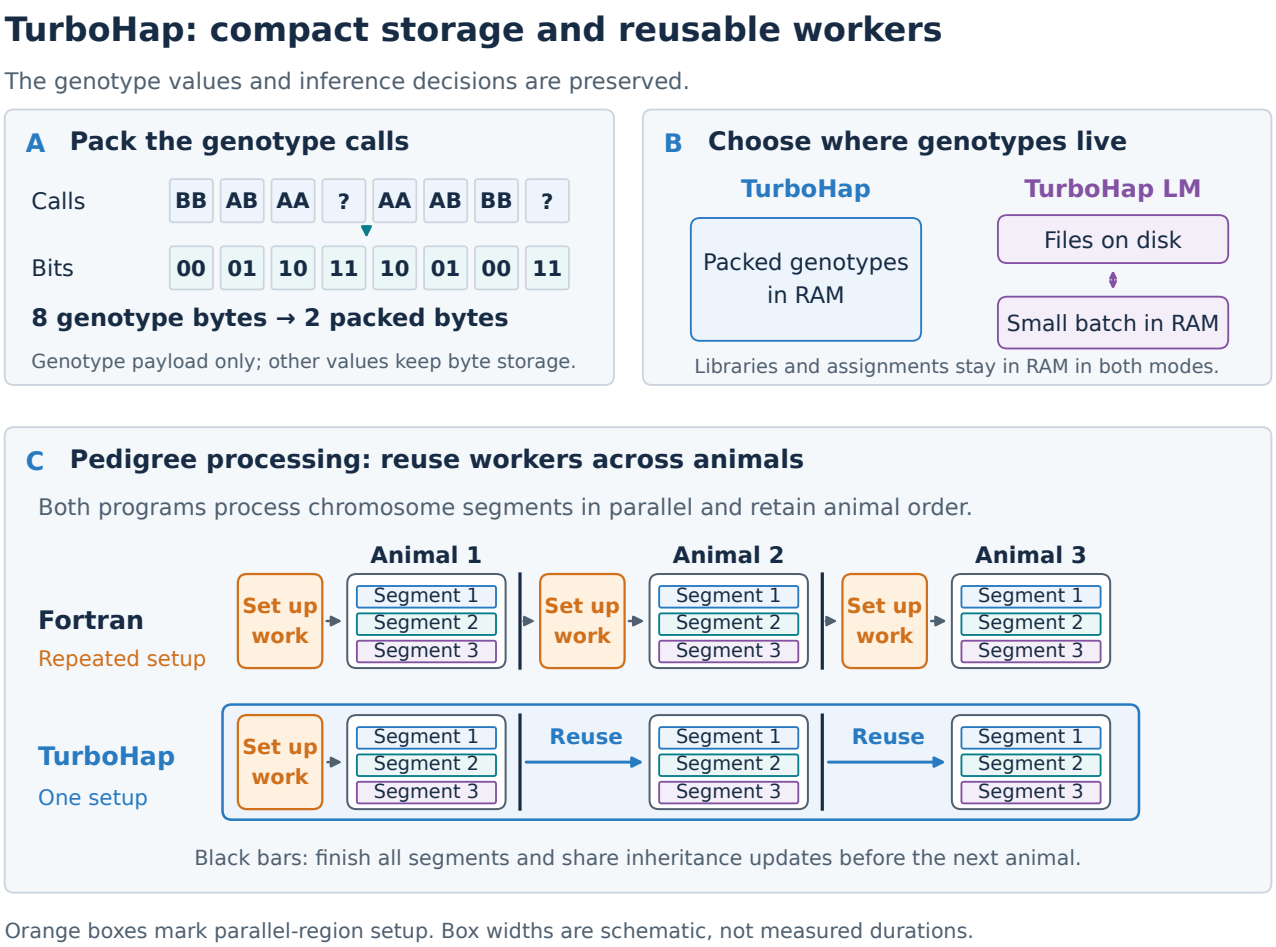


Figure 3. Compact genotype storage and reuse of parallel workers. (A) Two-bit encoding reduces eight ordinary genotype calls from eight bytes to two, excluding metadata and other structures. Values outside the supported four-state encoding retain byte storage. (B) Standard TurboHap keeps packed genotypes in RAM; LM transfers small batches between temporary files and RAM. Both retain haplotype libraries and assignments in memory. (C) During pedigree processing, both implementations process segments concurrently and preserve animal order. Fortran repeatedly enters parallel work regions; TurboHap keeps one worker team active across animals. Orange boxes indicate parallel-region setup. Black bars mark completion of segment work and inheritance updates before the next animal. Box widths do not represent measured durations.

Effects of dataset dimensions

Although the 10,000 × 100,000 and 100,000 × 10,000 datasets each contain one billion animal–locus combinations, standard TurboHap used 6.312 GiB on the marker-heavy dataset and 1.499 GiB on the animal-heavy dataset. More markers increase the number of segments and associated libraries, whereas more animals expand assignment and inheritance arrays within each segment. Together with configured capacity, these terms explain why matrix volume alone does not predict memory use.

Library sizes also varied: in the final step, positive-frequency haplotype counts ranged from 1,050 to 4,838 per segment for 100,000 × 10,000, compared with 3,226–19,861 for 1,000,000 × 10,000 (Supplementary Table S4). These final counts are not peak occupancy or candidate-search depth. Because pedigree structure, diversity and capacity vary along with dimensions, the comparisons describe these workloads rather than a general scaling law.

Discussion

TurboHap substantially reduced runtime while preserving FindHap's result files on the tested inputs. This required retaining the algorithm's choices throughout processing: the first acceptable haplotype, frequency ties, library updates and inheritance decisions can all influence later calls. Exact output comparison tests these choices together, including formatting.

The implementation avoids repeated input parsing, stores genotypes compactly and rejects incompatible candidates before a full comparison. Standard mode adds ordered lookup; persistent worker teams reduce parallel setup costs. The benchmarks measure their combined effect, including com-

Table 1. Complete-process runtime and speedup. Values are medians of two runs; displayed durations are rounded, while ratios use unrounded measurements. Bold denotes the lowest median runtime in each row and its corresponding speedup, not statistical significance.

Genotyped animals × loci	Fortran	TurboHap	Speedup	TurboHap LM	Speedup
10,000 × 1,000	10.52 s	**1.06 s**	**10.0**×	1.49 s	7.1×
10,000 × 10,000	1 min 16 s	**2.64 s**	**28.6**×	2.99 s	25.2×
10,000 × 100,000	7 min 28 s	**15.99 s**	**28.0**×	24.32 s	18.4×
100,000 × 1,000	14 min 34 s	**15.04 s**	**58.1**×	21.33 s	41.0×
100,000 × 10,000	19 min 42 s	**23.74 s**	**49.8**×	32.15 s	36.8×
100,000 × 100,000	1 h 17 min 0 s	**2 min 34 s**	**30.0**×	3 min 57 s	19.5×
1,000,000 × 10,000	6 h 53 min 40 s	**4 min 26 s**	**93.4**×	7 min 22 s	56.2×

Table 2. Peak process resident memory. Negative changes denote lower memory than the Fortran baseline. Values are medians of two runs, calculated independently of runtime medians. Bold denotes the lowest median RSS in each row and its corresponding percentage change. These measurements are not a total-system-memory bound.

Genotyped animals × loci	Fortran (GiB)	TurboHap (GiB)	Change	TurboHap LM (GiB)	Change
10,000 × 1,000	0.302	0.182	-39.6%	**0.180**	**-40.5%**
10,000 × 10,000	1.197	0.765	-36.1%	**0.746**	**-37.7%**
10,000 × 100,000	10.645	6.312	-40.7%	**5.910**	**-44.5%**
100,000 × 1,000	0.377	0.331	-12.2%	**0.316**	**-16.3%**
100,000 × 10,000	1.615	1.499	-7.2%	**1.101**	**-31.8%**
100,000 × 100,000	14.985	13.025	-13.1%	**8.581**	**-42.7%**
1,000,000 × 10,000	16.117	16.472	+2.2%	**12.077**	**-25.1%**

piler settings. Experiments that disable individual optimisations, together with phase profiling, would be needed to assign a share of the speedup to each change.

Packing and filtering reduce overhead but do not remove dependence on the number of animals, segments and distinct haplotypes. Assignment arrays and haplotype libraries remain resident in both modes, and dense output still requires work proportional to the number of animals times the number of markers written. LM trades additional I/O and temporary disk space for lower resident memory. Supplementary File 1 describes the main storage and search terms.

The simulation design limits extrapolation to production populations. Large families and a small ancestral population can affect haplotype diversity, which in turn influences library sizes and the candidates examined before a match is found. Recombination and mutation also generate distinct haplotypes, so founder count alone does not determine this workload. The retained logs quantify final library sizes, but not search depths or filter rejection rates; more diverse pedigrees would be needed to test how those differences affect speedup. Moreover, a fixed adjacent-marker linkage parameter does not preserve linkage disequilibrium over physical distance when marker density changes.

The system comparison is likewise specific to the measured builds and conditions: one ARM64 machine, two repeats, different compiler tuning and unrecorded concurrent server load. A matched-tuning Fortran build and further repetitions on an otherwise idle system would strengthen the comparison, while thread-scaling tests and controlled ablations would separate parallel and search-related gains. Comparing standard mode with LM cannot isolate the bitmap index, because their storage and I/O also differ. Peak RSS measures process residency rather than LM's temporary-disk requirements or all filesystem-cache costs, and the higher capacity on the largest dataset changes both storage width and library allocations.

The present evidence establishes output agreement on the tested cases, not an independent assessment of biological accuracy or a proof for every possible input. Further tests on diverse pedigrees and x86-64 hardware would establish how widely the performance gains extend. Within the workloads measured here, standard TurboHap provides the fastest execution and LM the lowest resident-memory use.

Software and data availability

TurboHap v1.0.0 is publicly available at https://github.com/adrian-greenneuron/TurboHap, with

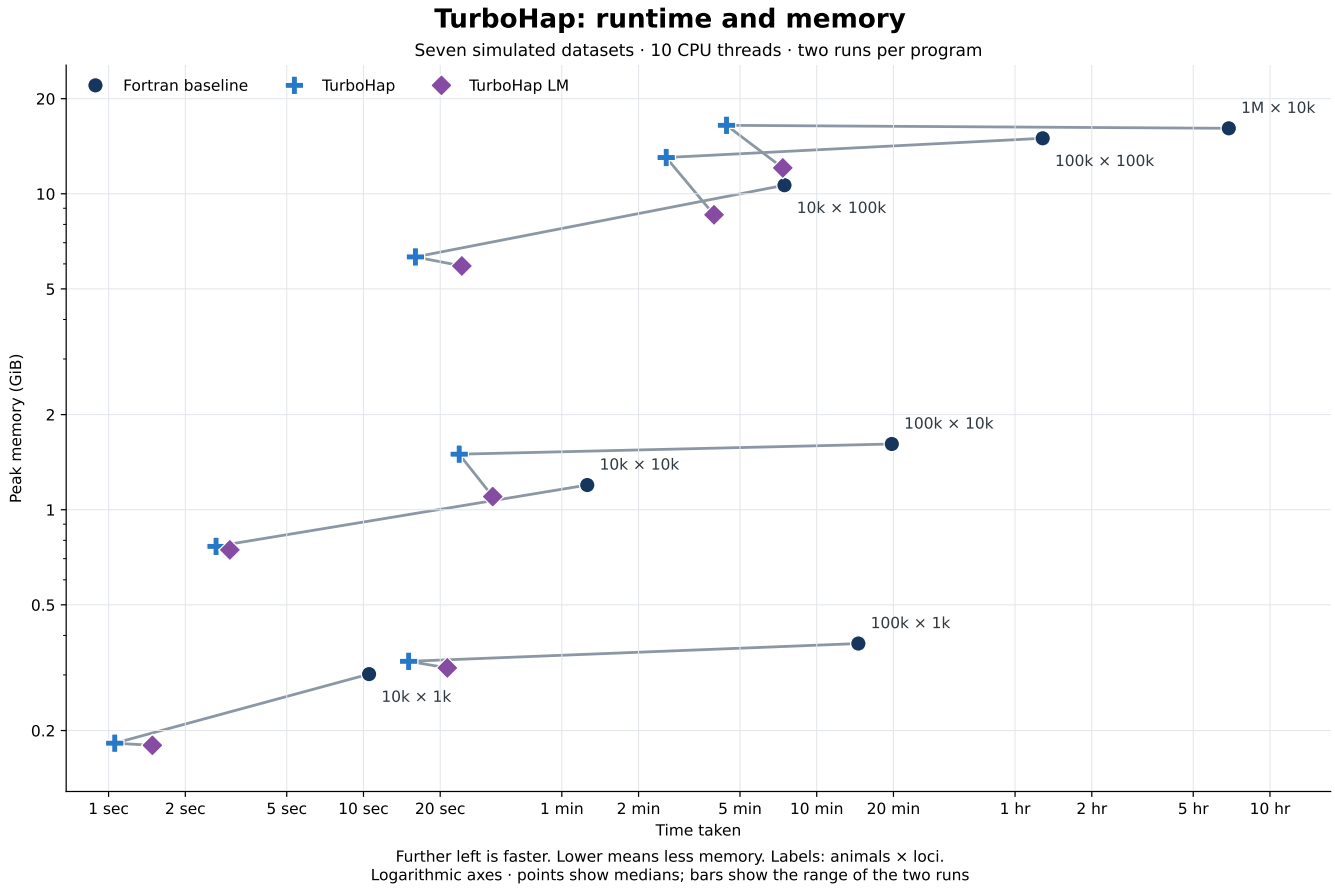


Figure 4. Runtime and peak resident memory. Points show separate median time and RSS values from two runs; bars show their ranges. Lines join programs on the same dataset. Both axes are logarithmic: further left is faster and lower uses less resident memory. Labels give genotyped animals × markers.

source code, installation instructions, the original FindHap example and benchmark results. Source and build identities are recorded in Supplementary File 2.

TurboHap is free for non-commercial academic research and teaching under its custom licence; commercial use requires a paid licence. A C++17 compiler, OpenMP and CMake are required. The executable is named `findhap`; `--low-memory` selects LM. Validation reported here used Linux ARM64 with GCC 13.3.

Supplementary File 1 contains detailed methods, all individual observations and the simulation recipe. Supplementary File 2 provides measurements, input and truth manifests, output-check records, reference sources, reproduction tools and figures. The release benchmark documentation reports the same results. All seven synthetic datasets and retained genotype and haplotype truth are available on Zenodo under CC BY 4.0 [6].

Declarations

Article licence. This article and its figures are licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). TurboHap software remains subject to its separate custom software licence.

Competing interests. Adrian R. Carr is affiliated with Green Neuron AI (Green Neuron Ltd), which licenses TurboHap for commercial use.

Funding. This work was funded by Green Neuron Ltd, trading as Green Neuron AI.

Author contributions. Adrian R. Carr is the sole author and contributed to all aspects of the work: conceptualisation, methodology, software, validation, formal analysis, investigation, resources, data curation, visualisation, writing the original draft, review and editing, project administration, and funding acquisition.

Acknowledgements. The author thanks Paul M. VanRaden and the coauthors and contributors to FindHap for the original method and reference software, and the developers of the USDA genosim simulation resources.

References

1. VanRaden PM, O'Connell JR, Wiggans GR, Weigel KA. Genomic evaluations with many more geno-

types. *Genetics Selection Evolution*. 2011;43:10. doi:10.1186/1297-9686-43-10.

2. VanRaden PM, Null DJ, Sargolzaei M, Wiggans GR, Tooker ME, Cole JB, Sonstegard TS, Connor EE, Winters M, van Kaam JBCHM, Valentini A, Van Doormaal BJ, Faust MA, Doak GA. Genomic imputation and evaluation using high-density Holstein genotypes. *Journal of Dairy Science*. 2013;96(1):668–678. doi:10.3168/jds.2012-5702.
3. VanRaden PM; USDA Agricultural Research Service. *FINDHAP: Find haplotypes and impute genotypes using multiple chip sets and sequence data*. Software documentation and v3 distribution. Accessed 20 October 2026. USDA FindHap.
4. Sargolzaei M, Chesnais JP, Schenkel FS. A new approach for efficient genotype imputation using information from relatives. *BMC Genomics*. 2014;15:478. doi:10.1186/1471-2164-15-478.
5. USDA Agricultural Research Service. *GENOSIM: Simulate genotypes*. Version 4 software distribution. Accessed 20 October 2026. USDA GENOSIM.
6. Carr AR. *TurboHap synthetic benchmark datasets and simulation truth*. Version 1.0.0. Zenodo; 2026. doi:10.5281/zenodo.23566926.